

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание №4

«Технический дизайн микросервисной архитектуры»

Выполнил:

Якшин Артемий

Группа БР1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Разделить монолитное бэкэнд-приложение «Restaurant Booking» (бронирование столиков в ресторанах, реализовано в LP1) на микросервисы по принципу *database-per-service*. Спроектировать микросервисную архитектуру: показать взаимосвязь сервисов и способы их взаимодействия; спроектировать разделение БД на обособленные базы; спроектировать и описать эндпоинты межсервисного взаимодействия в формате OpenAPI с примерами запросов/ответов и возможными ошибками. Результат — документ с конкретными требованиями и шагами перехода от монолита к набору микросервисов.

Ход работы

1. Анализ монолита и выделение ограниченных контекстов

Текущий монолит — Node.js + Express + TypeORM с одной БД (9 сущностей: `users`, `restaurants`, `cuisines`, `restaurant_cuisines`, `menu_items`, `restaurant_photos`, `restaurant_tables`, `reservations`, `reviews`). По функциональным требованиям (Д32) выделяются четыре устойчивых ограниченных контекста (bounded contexts), вокруг которых группируются данные и операции; их и выносим в отдельные сервисы. Перед ними ставится **API Gateway / BFF** — единая точка входа для клиентов: маршрутизация, проверка клиентского JWT (через Auth Service), агрегация, rate limiting, CORS.

2. Состав микросервисов

Сервис	Зона ответственности	Своя БД	Таблицы
Auth / User Service (порт 8081)	Регистрация, вход, выпуск и интроспекция JWT, профиль пользователя; выпуск сервисных токенов (client_credentials)	<code>auth_db</code>	<code>users</code>
Catalog Service (порт 8082)	Рестораны, кухни, меню, фотографии, столики; поиск и фильтрация ресторанов; выдача меню/фото/столиков	<code>catalog_db</code>	<code>restaurants</code> , <code>cuisines</code> , <code>restaurant_cuisines</code> , <code>menu_items</code> , <code>restaurant_photos</code> , <code>restaurant_tables</code>
Reservation Service (порт 8083)	Создание/изменение/отмена бронирований, проверка занятости столика, история бронирований пользователя	<code>reservations_db</code>	<code>reservations</code> + read-model <code>tables_replica</code> (table_id, restaurant_id, capacity)
Review Service (порт 8084)	Создание/редактирование/удаление отзывов, список отзывов ресторана, расчёт среднего рейтинга	<code>reviews_db</code>	<code>reviews</code>

Публичные клиентские маршруты из LP1 перераспределяются по сервисам так: `/auth/*`, `/users/*` → Auth; `/restaurants`, `/restaurants/{id}`, `/restaurants/{id}/{photos|menu|tables}`, `/cuisines` → Catalog; `/reservations*`, `/users/me/reservations` → Reservation; `/reviews*`, `/restaurants/{id}/reviews` → Review. Внешний контракт для клиента не меняется — маршрутизацию обеспечивает Gateway.

3. Архитектурная диаграмма

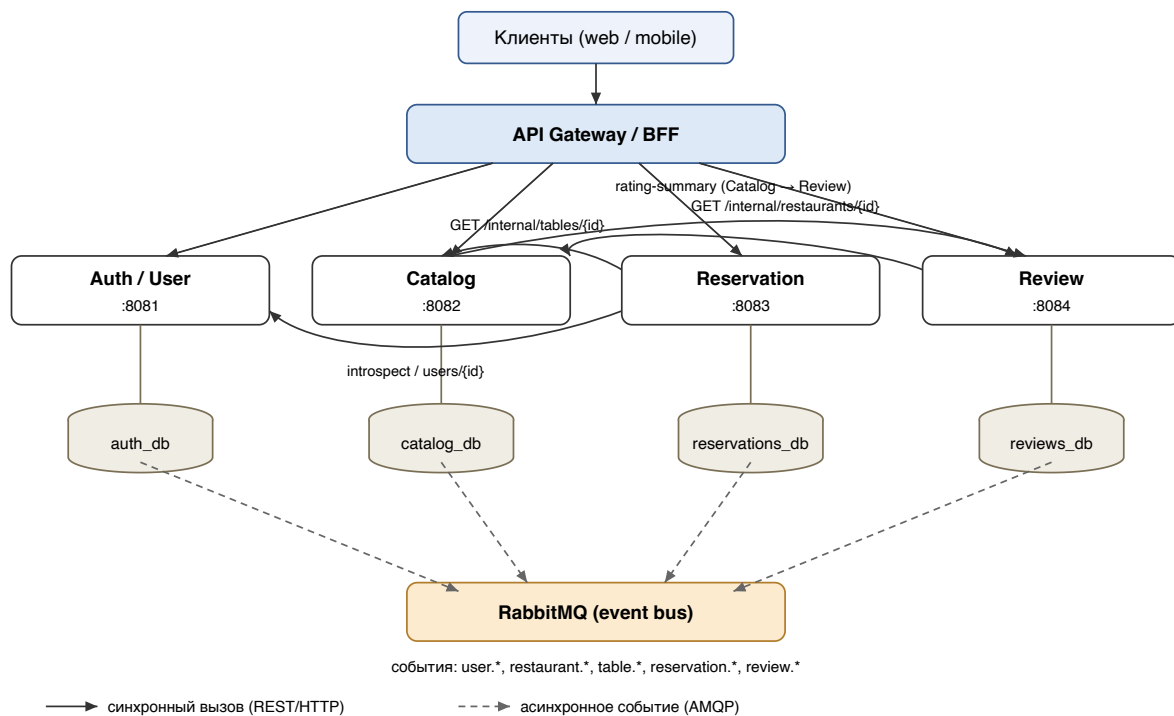


Рис. 1. Микросервисная архитектура «Restaurant Booking»

4. Способы взаимодействия сервисов

Синхронное (REST/HTTP, JSON). Применяется там, где нужен немедленный ответ при обработке клиентского запроса:

- Gateway → любой сервис — обслуживание клиентских запросов;
- любой сервис → **Auth**: `POST /internal/auth/introspect` (проверка клиентского токена, если Gateway передаёт его «как есть») или `GET /internal/users/{id}` / `POST /internal/users/batch` (обогащение ответов);
- **Reservation** → **Catalog**: `GET /internal/tables/{id}` — при создании брони проверить существование столика, его вместимость и ресторан-владелец;
- **Catalog** → **Reservation**: `GET /internal/tables/{id}/availability` / `POST /internal/reservations/availability-batch` — пометить столики `available/reserved` в публичном ответе `GET /restaurants/{id}/tables`;
- **Catalog** → **Review**: `GET /internal/restaurants/{id}/rating-summary` / `POST /internal/ratings/batch` — добавить `average_rating`, `reviews_count` в карточку/список ресторанов;
- **Review** → **Catalog**: `GET /internal/restaurants/{id}` — проверить существование ресторана перед созданием отзыва;
- **Review** → **Auth**: `POST /internal/users/batch` — обогатить список отзывов именами авторов.

Аутентификация межсервисных вызовов — заголовок `X-Service-Token` (короткоживущий служебный JWT от Auth по `client_credentials`, содержит claim `svc` и `scope'y`). Надёжность — таймауты, ретрай с экспоненциальной задержкой, `circuit breaker`; недоступность зависимого сервиса транслируется клиенту как `503`. Сквозной `correlation_id` прокидывается во всех вызовах для трассировки.

Асинхронное (события через RabbitMQ). Применяется для eventual-consistency, репликации read-моделей и развязки сервисов. Каждый сервис публикует доменные события в свой topic-exchange; заинтересованные сервисы подписываются. Доставка — *at-least-once*, потребители идемпотентны (дедупликация по `event_id`).

Событие	Издатель	Подписчики и реакция
<code>user.registered</code> , <code>user.updated</code>	Auth	Reservation, Review — обновить локальный кэш карточек пользователей (необязательно, опция вместо синхронного вызова)
<code>restaurant.created</code> , <code>restaurant.updated</code> , <code>restaurant.deleted</code>	Catalog	Review — кэш существования/названий ресторанов; Reservation — каскадные действия при удалении
<code>table.created</code> , <code>table.updated</code> , <code>table.deleted</code>	Catalog	Reservation — поддерживать read-model <code>tables_replica</code> (<code>id</code> , <code>restaurant_id</code> , <code>capacity</code>), чтобы не дёргать Catalog на каждую бронь
<code>reservation.created</code> , <code>reservation.cancelled</code>	Reservation	(на будущее) Notification/Analytics; Catalog — при необходимости инвалидация кэша доступности
<code>review.created</code> , <code>review.updated</code> , <code>review.deleted</code>	Review	Catalog — пересчитать/инвалидировать кэшированную сводку рейтинга ресторана

5. Разделение базы данных (database-per-service)

Единая БД монолита разрезается на четыре независимые базы — каждая принадлежит одному сервису, прямой доступ из других сервисов запрещён (только через API/события). Межтабличные связи, пересекающие границу сервиса, превращаются из FK в простые числовые ссылки + проверка через внутренний API и/или репликация необходимого минимума данных событиями.

База	Таблицы	Внешние ссылки (бывшие FK)	Как обеспечивается целостность
<code>auth_db</code>	<code>users</code>	—	Источник истины о пользователях
<code>catalog_db</code>	<code>restaurants</code> , <code>cuisines</code> , <code>restaurant_cuisines</code> , <code>menu_items</code> , <code>restaurant_photos</code> , <code>restaurant_tables</code>	—	Все FK внутри одного контекста — остаются обычными FK
<code>reservations_db</code>	<code>reservations</code> (<code>user_id</code> , <code>table_id</code> , <code>restaurant_id</code> — без FK), <code>tables_replica</code> (<code>table_id</code> PK, <code>restaurant_id</code> , <code>capacity</code>)	<code>user_id</code> → <code>auth</code> , <code>table_id/restaurant_id</code> → <code>catalog</code>	<code>user_id</code> берётся из проверенного JWT; <code>table_id</code> валидируется вызовом <code>GET /internal/tables/{id}</code> при создании; <code>tables_replica</code> наполняется событиями <code>table.*</code>
<code>reviews_db</code>	<code>reviews</code> (<code>user_id</code> , <code>restaurant_id</code> — без FK; уникальность пары сохраняется)	<code>user_id</code> → <code>auth</code> , <code>restaurant_id</code> → <code>catalog</code>	<code>restaurant_id</code> валидируется вызовом <code>GET /internal/restaurants/{id}</code> ; имена авторов берутся из <code>POST /internal/users/batch</code>

Замечание о согласованности. Между сервисами обеспечивается не строгая, а конечная согласованность (eventual consistency). Операции, затрагивающие несколько сервисов (см. §6.3), реализуются как saga с

компенсациями; «осиротевшие» ссылки (например, отзыв на удалённый ресторан) обрабатываются обработчиками соответствующих событий.

6. Эндпоинты межсервисного взаимодействия

Полная спецификация — в файле `homeworks/hw4/openapi-internal.yaml` (OpenAPI 3.0.3). Эти эндпоинты не выставляются наружу через Gateway, доступны только внутри периметра и требуют заголовков `X-Service-Token`. Сводка:

Сервис	Метод и путь	Назначение
Auth	POST <code>/internal/auth/introspect</code>	Проверка клиентского JWT → {active, user_id, email, expires_at}
	GET <code>/internal/users/{userId}</code>	Краткая карточка пользователя
	POST <code>/internal/users/batch</code>	Пакет карточек пользователей по списку id
Catalog	GET <code>/internal/restaurants/{restaurantId}</code>	Краткая карточка ресторана (+ проверка существования)
	POST <code>/internal/restaurants/batch</code>	Пакет карточек ресторанов
	GET <code>/internal/tables/{tableId}</code>	Карточка столика: restaurant_id, table_number, capacity
Reservation	GET <code>/internal/tables/{tableId}/availability?date&time</code>	Свободен ли столик на дату/время
	POST <code>/internal/reservations/availability-batch</code>	Доступность набора столиков на дату/время
Review	GET <code>/internal/restaurants/{restaurantId}/rating-summary</code>	{restaurant_id, average_rating, reviews_count}
	POST <code>/internal/ratings/batch</code>	Сводки рейтингов для списка ресторанов

6.1. Пример запроса/ответа

```
// Reservation Service → Catalog Service при создании брони
GET /internal/tables/5 HTTP/1.1
Host: catalog-service:8082
X-Service-Token: eyJhbGciOiJIUzI1Ni... (svc=reservation-service, scope=catalog.read)
X-Correlation-Id: 7ble...

HTTP/1.1 200 OK
Content-Type: application/json
{ "table_id": 5, "restaurant_id": 1, "table_number": 5, "capacity": 4 }
```

```
// Catalog Service → Review Service при сборке списка ресторанов
POST /internal/ratings/batch HTTP/1.1
Host: review-service:8084
X-Service-Token: eyJhbGciOiJIUzI1Ni... (svc=catalog-service, scope=reviews.read)
{ "restaurant_ids": [1, 2, 3, 4] }
```

```
HTTP/1.1 200 OK
[ { "restaurant_id": 1, "average_rating": 4.5, "reviews_count": 2 },
  { "restaurant_id": 2, "average_rating": 0, "reviews_count": 0 },
  { "restaurant_id": 3, "average_rating": 5.0, "reviews_count": 1 },
  { "restaurant_id": 4, "average_rating": 0, "reviews_count": 0 } ]
```

6.2. Возможные ошибки межсервисных вызовов

Код	Когда	Тело
-----	-------	------

400 Bad Request	Некорректные параметры/тело (например, пустой <code>user_ids</code> , неверный формат <code>date</code>)	<code>{ "error": "Bad Request", "message": "...", "correlation_id": "..." }</code>
401 Unauthorized	Отсутствует/просрочен/неподходящий <code>X-Service-Token</code> (нет нужного <code>scope</code>)	<code>{ "error": "Unauthorized", "message": "Invalid or missing X-Service-Token" }</code>
404 Not Found	Запрошенный ресурс не существует (нет столика/ресторана/пользователя)	<code>{ "error": "Not Found", "message": "Resource not found" }</code>
409 Conflict	Нарушение бизнес-инварианта при записи (например, при синхронном бронировании — столик уже занят)	<code>{ "error": "Conflict", "message": "Table is already booked for this date and time" }</code>
503 Service Unavailable	Зависимый сервис недоступен/таймаут/сработал <code>circuit breaker</code> — вызывающий транслирует это клиенту	<code>{ "error": "Service Unavailable", "message": "catalog-service is unavailable" }</code>

6.3. Сквозные сценарии

Создание бронирования (клиент → Gateway → Reservation Service):

1. Gateway проверяет клиентский JWT (или Reservation вызывает `POST /internal/auth/introspect`), получает `user_id`;
2. Reservation: `GET /internal/tables/{table_id}` у Catalog → существование столика, `capacity`, `restaurant_id`; если 404 — клиенту 404, если `guests_count > capacity` — 400;
3. Reservation в своей БД проверяет, что на (`table_id`, `date`, `time`) нет активной брони; если есть — 409;
4. Reservation создаёт запись `reservations` (`status = confirmed`) и публикует `reservation.created`;
5. клиенту возвращается бронь; данные ресторана для ответа берутся из шага 2 (или дозапрашиваются у Catalog).

Если бы шаги затрагивали запись в нескольких сервисах, использовалась бы `saga`: на ошибку последующего шага — компенсирующее действие (например, отмена созданной брони и событие `reservation.cancelled`).

Список ресторанов с фильтрами (клиент → Gateway → Catalog Service):

1. Catalog выполняет фильтрацию/пагинацию по `catalog_db`;
2. для полученной страницы ресторанов — `POST /internal/ratings/batch` у Review (или берёт кэш, наполняемый событиями `review.*`), добавляет `average_rating/reviews_count`;
3. возвращает клиенту ответ в том же формате, что и монолит в ЛР1.

7. Конкретные требования и шаги перехода

1. **Зафиксировать границы.** 4 сервиса (Auth, Catalog, Reservation, Review) + API Gateway; перечень публичных маршрутов на каждый сервис (см. §2).
2. **Разделить кодовую базу.** Вынести каждый контекст в отдельный модуль/репозиторий с собственным `package.json`, Express-приложением, набором сущностей TypeORM и `DataSource`.
3. **Разделить БД.** Создать `auth_db`, `catalog_db`, `reservations_db`, `reviews_db`; перенести соответствующие таблицы; **убрать межконтекстные FK**, заменив их числовыми ссылками; в `reservations_db` добавить таблицу `tables_replica`.

4. **Внедрить API Gateway.** Маршрутизация на сервисы по префиксу пути; проверка клиентского JWT (через Auth introspection или общий секрет); проксирование `Authorization/X-Correlation-Id`.
5. **Сервисная аутентификация.** Реализовать в Auth выпуск служебных токенов (`client_credentials`) и middleware проверки `x-Service-Token` + score во всех сервисах.
6. **Реализовать внутренние REST-эндпоинты** по спецификации `openapi-internal.yaml` и HTTP-клиенты для межсервисных вызовов с таймаутами/ретраями/circuit breaker.
7. **Поднять брокер сообщений (RabbitMQ).** Завести exchange'ы на каждый сервис-издатель; реализовать публикацию доменных событий и идемпотентных потребителей (дедупликация по `event_id`); наполнять `tables_replica` и кэш рейтингов событиями.
8. **Перенести бизнес-логику** из монолита в сервисы; для операций, затрагивающих несколько сервисов, оформить saga с компенсациями.
9. **Наблюдаемость.** Сквозной `correlation_id`, структурное логирование, базовые метрики и health-checks (`/health`) на каждом сервисе.
10. **Контейнеризация.** Dockerfile на каждый сервис + `docker-compose.yaml` (4 сервиса + Gateway + 4 БД + RabbitMQ) — задел под ЛР2/ЛР3.
11. **Стратегия миграции.** Strangler-подход: сначала Gateway проксирует всё в монолит; затем по одному контексту трафик переключается на новый сервис, его данные мигрируются и удаляются из БД монолита; в конце монолит выводится из эксплуатации.

Вывод

Монолитное приложение «Restaurant Booking» декомпозировано на четыре микросервиса (Auth/User, Catalog, Reservation, Review) за единым API Gateway, каждый — со своей базой данных (database-per-service). Спроектированы способы взаимодействия: синхронные REST-вызовы с сервисным токеном для немедленных проверок/обогащения и асинхронные события через RabbitMQ для репликации read-моделей и развязки сервисов. Описано разрезание единой БД на четыре независимые базы с заменой межконтекстных внешних ключей на числовые ссылки, валидируемые через API и поддерживаемые событиями. Эндпоинты межсервисного взаимодействия специфицированы в формате OpenAPI (файл `openapi-internal.yaml`) с примерами запросов/ответов и каталогом ошибок (400/401/404/409/503). Сформирован перечень конкретных требований и пошаговый план перехода от монолита к микросервисной архитектуре по принципу strangler, что служит основой для ЛР2 (реализация микросервисов).